

Análisis Predictivo: Maratón de Boston (2015-2017)

Aprendizaje Automático | Máster en Análisis de Datos Deportivos

Alba Martínez de la Hermosa Daniel Lopez Paredes
Alonso González Romero

5 de May, 2026

Table of contents

1	Introducción	3
1.1	Resumen	3
2	Análisis Exploratorio de Datos	4
2.1	Importación de librerías	4
2.2	Carga de datos	4
2.3	Estructura del conjunto de datos	5
2.3.1	1. Datos Sociodemográficos y de Origen	5
2.3.2	2. Tiempos de Parciales (<i>Splits</i>)	6
2.3.3	3. Rendimiento y Clasificación Final	7
2.4	Integración y Consolidación del Conjunto de Datos	8
2.5	Preprocesamiento y Limpieza de Variables	9
2.6	Análisis de datos nulos	12
2.7	Incorporación de Variables Meteorológicas	13
2.8	Categorización por Nivel de Rendimiento	14
2.9	Definición de la Variable de Fatiga Extrema (<i>Hitting the Wall</i>)	16

1 Introducción

1.1 Resumen

Este trabajo presenta un flujo completo de Ciencia de Datos aplicado a los resultados del **Maratón de Boston** en sus ediciones 2015, 2016 y 2017.

El objetivo principal es desarrollar modelos de **Aprendizaje Automático** capaces de predecir el tiempo de finalización de los corredores basándonos en variables de rendimiento parcial, condiciones externas de carrera y variables sociodemográficas.

(rellenar más cuando tengamos todo el trabajo hehco)

2 Análisis Exploratorio de Datos

En este primer capítulo se realizará un análisis exploratorio de datos (EDA) con el doble objetivo de comprender la estructura interna de los datasets actuales y de añadir variables externas (meteorológicas y temporales) que resulten significativas para el posterior entrenamiento de nuestros modelos.

Aunque el núcleo de este trabajo es el modelado predictivo, la fase de exploración y preprocesamiento representa el pilar fundamental que garantiza la calidad y robustez de cualquier sistema de aprendizaje automático.

2.1 Importación de librerías

Para comenzar, cargamos el ecosistema básico de librerías en Python para realizar dicho trabajo de exploración y preprocesamiento.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Configuración visual para que los gráficos queden de revista
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
```

2.2 Carga de datos

Disponemos de tres archivos independientes en formato CSV que contienen los resultados oficiales de la Maratón de Boston para las ediciones de 2015, 2016 y 2017. Procedemos a su lectura para poder posteriormente empezar a realizar los diferentes análisis.

```
df_2015 = pd.read_csv('../data/raw/results2015.csv')
df_2016 = pd.read_csv('../data/raw/results2016.csv')
df_2017 = pd.read_csv('../data/raw/results2017.csv')
```

2.3 Estructura del conjunto de datos

Una vez tenemos los tres conjuntos de datos, vamos a realizar un pequeño estudio de la estructura que tienen, para ello, vamos a comenzar revisando si tienen exactamente las mismas variables:

```
# Comprobamos si las columnas y su orden son idénticos
todo_igual = df_2015.columns.equals(df_2016.columns) and df_2016.columns.equals(df_2017.columns)

# Obtenemos el número de columnas de cada dataframe
num_cols_15 = len(df_2015.columns)
num_cols_16 = len(df_2016.columns)
num_cols_17 = len(df_2017.columns)

# Imprimimos el resultado con el formato que quieres
if todo_igual:
    print(f"¿Son exactamente iguales en columnas y orden?: Sí, y además tienen {num_cols_15} columnas.")
else:
    print(f"¿Son exactamente iguales en columnas y orden?: No, tienen {num_cols_15}, {num_cols_16} y {num_cols_17} columnas.")
```

¿Son exactamente iguales en columnas y orden?: Sí, y además tienen 29 columnas.

Por lo tanto, tenemos 3 conjuntos de datos iguales y que además tienen 29 columnas. A continuación, vamos a elaborar en una tabla markdown, el significado de cada una.

El dataset original de la Maratón de Boston (ediciones 2015, 2016 y 2017) cuenta con un conjunto de variables que combinan datos sociodemográficos, tiempos de paso parciales y clasificaciones finales. A continuación se detalla el significado de cada columna estructurado por categorías:

2.3.1 1. Datos Sociodemográficos y de Origen

Variable	Tipo de Dato	Descripción
bib	string	Número de dorsal del corredor. <i>Nota:</i> Puede contener caracteres no numéricos en atletas de élite o categorías especiales (ej. F1, W10).

Variable	Tipo de Dato	Descripción
name	Object	Nombre completo del atleta. Existen otras 2 columnas que dividen este nombre en nombre y apellidos
age	float64	Edad del corredor el día de la carrera.
gender	string	Género del corredor (M para masculino, F para femenino).
city	string	Ciudad de residencia registrada por el participante.
state	string	Estado o provincia de residencia (principalmente para corredores de EE. UU. y Canadá).
country_residence	string	Código de 3 letras que representa el país de origen del corredor (ej. USA, ESP, MEX).
contry_citizenship	string	Nacionalidad o país de ciudadanía del corredor (suele estar vacío si coincide con el país de residencia).

2.3.2 2. Tiempos de Parciales (*Splits*)

Estas columnas registran el tiempo transcurrido acumulado desde el pistoletazo de salida (o cruce de la línea de salida) hasta cada punto de control específico.

Variable	Tipo de Dato	Punto de Control	Distancia Acumulada
5K	string	Tiempo de paso por el kilómetro 5.	5,0 km
10K	string	Tiempo de paso por el kilómetro 10.	10,0 km

Variable	Tipo de Dato	Punto de Control	Distancia Acumulada
15K	string	Tiempo de paso por el kilómetro 15.	15,0 km
20K	string	Tiempo de paso por el kilómetro 20.	20,0 km
Half	string	Tiempo de paso por la Media Maratón.	21,097 km
25K	string	Tiempo de paso por el kilómetro 25.	25,0 km
30K	string	Tiempo de paso por el kilómetro 30.	30,0 km
35K	string	Tiempo de paso por el kilómetro 35.	35,0 km
40K	string	Tiempo de paso por el kilómetro 40.	40,0 km

2.3.3 3. Rendimiento y Clasificación Final

Variable	Tipo de Dato	Descripción
Pace	string	Ritmo medio por milla de toda la carrera en formato MM:SS.
projected_time	string	Tiempo final estimado o proyectado (generalmente equivalente al tiempo oficial).
official_time	Object	Tiempo neto oficial de meta (Chip Time). Representa el tiempo real de carrera desde que el atleta cruza la línea de salida hasta que pisa la de meta. (Variable
overall	Integer	Objetivo / Target Y). Posición en la clasificación general absoluta de la carrera.

Variable	Tipo de Dato	Descripción
gender_result	Integer	Posición en la clasificación general de su respectivo género.
division_result	Integer	Posición en la clasificación de su grupo de edad específico.
seconds	Integer	Segundos tardados en completar la maratón

2.4 Integración y Consolidación del Conjunto de Datos

Para poder entrenar y validar nuestros modelos de aprendizaje automático de forma global, es necesario unificar las tres fuentes de datos (`df_2015`, `df_2016` y `df_2017`) en un único dataframe consolidado.

Antes de realizar la concatenación, añadiremos la variable **Date** con la fecha exacta de realización de cada edición. La Maratón de Boston se celebra históricamente el tercer lunes de abril (Patriots' Day), correspondiendo a las siguientes fechas reales: * **Edición 2015:** 20 de abril de 2015 (2015-04-20) * **Edición 2016:** 18 de abril de 2016 (2016-04-18) * **Edición 2017:** 17 de abril de 2017 (2017-04-17)

Al añadir esta variable temporal no solo mantenemos la trazabilidad del año de origen de cada registro, sino que habilitamos la posibilidad de cruzar estos datos en el futuro con variables externas como la meteorología del día exacto del evento.

```
# 1. Añadimos la columna 'Date' a cada dataframe individual en formato string
df_2015['Date'] = '2015-04-20'
df_2016['Date'] = '2016-04-18'
df_2017['Date'] = '2017-04-17'

# 2. Concatenamos los tres conjuntos de datos verticalmente
# ignore_index=True evita que los índices repetidos (0, 1, 2...) de cada año colisionen
df_boston = pd.concat([df_2015, df_2016, df_2017], ignore_index=True)

# 3. Convertimos la columna 'Date' a tipo datetime oficial de Pandas
df_boston['Date'] = pd.to_datetime(df_boston['Date'])

# 4. Verificamos las dimensiones finales del dataset unificado
print(f"Dimensiones del dataset unificado final: {df_boston.shape}")
```



```
print(f"Número total de registros integrados: {df_boston.shape[0]} corredores.")
print(f"Número total de columnas: {df_boston.shape[1]}")
```

Dimensiones del dataset unificado final: (79640, 30)
 Número total de registros integrados: 79640 corredores.
 Número total de columnas: 30

Una vez realizada la unificación, verificamos los datos realizando un muestreo de algunas columnas:

```
# Seleccionamos una muestra representativa con columnas demográficas y la nueva variable Date
columnas_muestra = ['name', 'age', 'gender', 'country_residence', 'official_time', 'Date']
df_muestra = df_boston[columnas_muestra].sample(5, random_state=42)

# Imprimimos la muestra formateada en markdown limpio
print(df_muestra.to_markdown(index=False))
```

name	age	gender	country_residence	official_time	Date
Loaiza, David	48	M	USA	3:24:27	2017-04-17 00:00:00
Fulfer, Clay	52	M	USA	5:14:15	2016-04-18 00:00:00
Meadows, Karen L	52	F	USA	4:00:58	2016-04-18 00:00:00
Johnson, David T	32	M	USA	2:54:17	2015-04-20 00:00:00
Paine, A. R.	51	M	USA	3:53:52	2016-04-18 00:00:00

2.5 Preprocesamiento y Limpieza de Variables

Una vez consolidado nuestro conjunto de datos, es fundamental adecuar los tipos de datos de las variables y descartar aquellas columnas redundantes o irrelevantes (como identificadores de nombres alternativos, sufijos o formatos duplicados) antes de alimentar nuestros algoritmos de Aprendizaje Automático.

En esta fase de preprocesamiento realizaremos las siguientes acciones: 1. **Selección de características (*Feature Selection*)**: Conservaremos únicamente el subconjunto de variables

demográficas, espaciales y de rendimiento necesarias, descartando variables de texto libre como `name` o redundantes como `seconds`. 2. **Corrección de tipos de datos:** Convertiremos la edad (`age`) de formato decimal a entero, y las variables de texto repetitivo como `gender` y `country_residence` a tipo categórico. 3. **Conversión temporal (Formatos HH:MM:SS a segundos):** Transformaremos todos los parciales de paso (5k, 10k, etc.) y la variable objetivo (`official_time`) a segundos totales, permitiendo que los modelos matemáticos puedan procesar estas distancias como variables numéricas continuas. 4. **Tratamiento de la fecha:** Simplificaremos la columna `Date` para conservar únicamente la fecha del evento sin la hora.

```
# 1. Definimos el subconjunto de variables que queremos conservar
columnas_conservar = [
    'bib', 'display_name', 'age', 'gender', 'city', 'country_residence',
    '5k', '10k', '15k', '20k', 'half', '25k', '30k', '35k', '40k',
    'pace', 'official_time', 'overall', 'gender_result', 'division_result', 'Date'
]

# Filtramos el dataframe original
df_boston = df_boston[columnas_conservar].copy()

# 2. Transformación de tipos básicos
# Convertimos la edad a entero de forma segura (eliminando previamente nulos si existieran en el dataframe)
df_boston = df_boston.dropna(subset=['age'])
df_boston['age'] = df_boston['age'].astype(int)

# Convertimos a tipo categórico para optimizar memoria y compatibilidad
df_boston['gender'] = df_boston['gender'].astype('category')
df_boston['country_residence'] = df_boston['country_residence'].astype('category')

# Forzamos que las variables identificadoras queden como texto limpio
df_boston['bib'] = df_boston['bib'].astype(str)
df_boston['display_name'] = df_boston['display_name'].astype(str)
df_boston['city'] = df_boston['city'].astype(str)

# 3. Función robusta para convertir "HH:MM:SS" a segundos totales
def tiempo_a_segundos(valor):
    if pd.isna(valor):
        return np.nan

    val_str = str(valor).strip()
    # Tratamos nulos ocultos en el formato original de la maratón
    if val_str in ('-', '', 'NaN', 'nan'):
        return np.nan
```

```

try:
    partes = val_str.split(':')
    if len(partes) == 3: # Formato HH:MM:SS
        return int(partes[0]) * 3600 + int(partes[1]) * 60 + int(partes[2])
    elif len(partes) == 2: # Formato MM:SS (por si acaso en algún parcial corto)
        return int(partes[0]) * 60 + int(partes[1])
    return float(val_str)
except ValueError:
    return np.nan

# Columnas temporales que pasaremos a segundos
columnas_tiempo = ['5k', '10k', '15k', '20k', 'half', '25k', '30k', '35k', '40k', 'official_time']

# Aplicamos la conversión a segundos
for col in columnas_tiempo:
    df_boston[col] = df_boston[col].apply(tiempo_a_segundos)

# 4. Modificar la variable Date para dejarla únicamente como fecha sin hora
df_boston['Date'] = pd.to_datetime(df_boston['Date']).dt.date

# 5. Comprobación final de los tipos de datos resultantes
print("Tipos de datos finales del dataset preprocesado:")
print(df_boston.dtypes)

```

Tipos de datos finales del dataset preprocesado:

bib	object
display_name	object
age	int64
gender	category
city	object
country_residence	category
5k	float64
10k	float64
15k	float64
20k	float64
half	float64
25k	float64
30k	float64
35k	float64
40k	float64
pace	object
official_time	int64

```

overall                float64
gender_result          float64
division_result        float64
Date                   object
dtype: object

```

2.6 Análisis de datos nulos

Una vez tenemos todos los datos enriquecidos con su formato ideal y las variables necesarias para realizar los modelos, ahora se va a revisar qué datos nulos tenemos y en qué variables. Para ello, comenzamos viendo en qué variables hay datos nulos y el número:

```

# 1. Calculamos las métricas de nulos por columna
total_registros = len(df_boston)
nulos_por_columna = df_boston.isnull().sum()
porcentaje_nulos = (nulos_por_columna / total_registros) * 100

# 2. Construimos un DataFrame resumen con los resultados
df_resumen_nulos = pd.DataFrame({
    'Variable': nulos_por_columna.index,
    'Valores Nulos': nulos_por_columna.values,
    'Total Registros': total_registros,
    'Porcentaje Nulos (%)': porcentaje_nulos.values
})

# Redondeamos el porcentaje a dos decimales para una visualización más limpia
df_resumen_nulos['Porcentaje Nulos (%)'] = df_resumen_nulos['Porcentaje Nulos (%)'].round(2)

# 3. FILTRADO: Nos quedamos únicamente con las variables que tengan más de 0 nulos
df_resumen_nulos = df_resumen_nulos[df_resumen_nulos['Valores Nulos'] > 0]

# 4. Imprimimos el resultado en formato Markdown utilizando tabulate de fondo
print(df_resumen_nulos.to_markdown(index=False))

```

Variable	Valores Nulos	Total Registros	Porcentaje Nulos (%)
5k	229	79638	0.29
10k	114	79638	0.14
15k	51	79638	0.06
20k	85	79638	0.11
half	62	79638	0.08

25k		81		79638		0.1	
30k		88		79638		0.11	
35k		86		79638		0.11	
40k		76		79638		0.1	

Observamos que hay un porcentaje ínfimo de datos nulos, es por ello que se decide eliminar aquellas filas que tengan algún dato nulo en los parciales.

2.7 Incorporación de Variables Meteorológicas

El rendimiento en una maratón está fuertemente condicionado por factores ambientales externos. Para que nuestros modelos de aprendizaje automático puedan capturar este impacto, añadiremos tres variables climatológicas clave asociadas a la fecha de cada edición:

1. **temp_mean_c**: Temperatura promedio durante las horas de la carrera (en grados Celsius).
2. **humidity_pct**: Porcentaje de humedad relativa promedio.
3. **wind_speed_kmh**: Velocidad promedio del viento (factor crítico en Boston debido a su recorrido lineal, donde el viento de cara actúa como un freno constante).

Las condiciones registradas para cada edición fueron: * **2015 (Frío, lluvia y viento de cara extremo)**: 8 °C, 93% de humedad, viento de 22 km/h. * **2016 (Clima templado/cálido y seco)**: 16 °C, 28% de humedad, viento de 13 km/h. * **2017 (Calor sofocante, adverso para larga distancia)**: 21 °C, 35% de humedad, viento de 11 km/h.

```
# 1. Definimos el diccionario con los datos climatológicos por fecha (Date)
# Usamos strings de fecha porque en el paso anterior convertimos 'Date' a tipo datetime/date
clima_datos = {
    '2015-04-20': {'temp_mean_c': 8.0, 'humidity_pct': 93, 'wind_speed_kmh': 22},
    '2016-04-18': {'temp_mean_c': 16.0, 'humidity_pct': 28, 'wind_speed_kmh': 13},
    '2017-04-17': {'temp_mean_c': 21.0, 'humidity_pct': 35, 'wind_speed_kmh': 11}
}

# Convertimos las llaves del diccionario a objetos datetime.date para asegurar que coincidan
import datetime
clima_datos_fechas = {datetime.datetime.strptime(k, "%Y-%m-%d").date(): v for k, v in clima_datos.items()}

# 2. Mapeamos las nuevas columnas en función de la fecha de la carrera
df_boston['temp_mean_c'] = df_boston['Date'].map(lambda x: clima_datos_fechas[x]['temp_mean_c'])
df_boston['humidity_pct'] = df_boston['Date'].map(lambda x: clima_datos_fechas[x]['humidity_pct'])
df_boston['wind_speed_kmh'] = df_boston['Date'].map(lambda x: clima_datos_fechas[x]['wind_speed_kmh'])
```

```
# 3. Comprobamos que se han añadido correctamente
print("Dataset con variables climatológicas integradas:")
print(df_boston[['Date', 'temp_mean_c', 'humidity_pct', 'wind_speed_kmh']].drop_duplicates())
```

Dataset con variables climatológicas integradas:

Date	temp_mean_c	humidity_pct	wind_speed_kmh
2015-04-20	8	93	22
2016-04-18	16	28	13
2017-04-17	21	35	11

2.8 Categorización por Nivel de Rendimiento

Con el objetivo de enriquecer el análisis descriptivo y facilitar posteriores estrategias de muestreo estratificado o validación cruzada controlada, procedemos a crear una nueva variable categórica ordinal que clasifique el nivel de rendimiento de cada atleta.

Para definir los umbrales de rendimiento, adaptamos el criterio oficial de la World Athletics y los tiempos de corte de los cajones de salida de las *World Marathon Majors* (como Boston). Dado que los ritmos basales difieren fisiológicamente por sexo, establecemos reglas diferenciadas para hombres (M) y mujeres (F):

- **Élite:**
 - Hombres: < 2h 17m 00s (< 8220 segundos)
 - Mujeres: < 2h 43m 00s (< 9780 segundos)
- **Alto nivel:**
 - Hombres: [2h 17m 00s - 2h 30m 00s) (8220 a < 9000 segundos)
 - Mujeres: [2h 43m 00s - 3h 00m 00s) (9780 a < 10800 segundos)
- **Muy entrenado/a:**
 - Hombres: [2h 30m 00s - 3h 00m 00s) (9000 a < 10800 segundos)
 - Mujeres: [3h 00m 00s - 3h 30m 00s) (10800 a < 12600 segundos)
- **Moderadamente entrenado/a:**
 - Hombres: [3h 00m 00s - 3h 30m 00s) (10800 a < 12600 segundos)
 - Mujeres: [3h 30m 00s - 4h 00m 00s) (12600 a < 14400 segundos)
- **Principiante:**
 - Hombres: ≥ 3h 30m 00s (≥ 12600 segundos)

– Mujeres: $\geq 4\text{h } 00\text{m } 00\text{s}$ (≥ 14400 segundos)

A continuación, implementamos una función de clasificación y la aplicamos sobre nuestro dataset unificado.

```
def asignar_categoria_rendimiento(row):
    genero = row['gender']
    tiempo = row['official_time']

    # Si por alguna razón el tiempo es nulo, devolvemos nulo
    if pd.isna(tiempo):
        return np.nan

    if genero == 'M':
        if tiempo < 8220:
            return 'Élite'
        elif tiempo < 9000:
            return 'Alto nivel'
        elif tiempo < 10800:
            return 'Muy entrenado/a'
        elif tiempo < 12600:
            return 'Moderadamente entrenado/a'
        else:
            return 'Principiante'

    elif genero == 'F':
        if tiempo < 9780:
            return 'Élite'
        elif tiempo < 10800:
            return 'Alto nivel'
        elif tiempo < 12600:
            return 'Muy entrenado/a'
        elif tiempo < 14400:
            return 'Moderadamente entrenado/a'
        else:
            return 'Principiante'

    else:
        # En caso de que exista otra etiqueta o nulo en género
        return 'No clasificado'

# 1. Aplicamos la función fila por fila
df_boston['athlete_level'] = df_boston.apply(asignar_categoria_rendimiento, axis=1)
```

```
# 2. Convertimos la columna a tipo categórica ordenada (de menor a mayor rendimiento)
orden_categorias = ['Principiante', 'Moderadamente entrenado/a', 'Muy entrenado/a', 'Alto nivel']
df_boston['athlete_level'] = pd.Categorical(df_boston['athlete_level'], categories=orden_categorias)

# 3. Comprobamos la distribución de nuestros corredores por nivel de rendimiento
distribucion_niveles = df_boston['athlete_level'].value_counts().reindex(orden_categorias).to_dict()
distribucion_niveles.columns = ['Número de Corredores']
distribucion_niveles['Porcentaje (%)'] = (distribucion_niveles['Número de Corredores'] / len(df_boston))

# Imprimimos la tabla de distribución en formato Markdown
print(distribucion_niveles.to_markdown())
```

athlete_level	Número de Corredores	Porcentaje (%)
Principiante	39845	50.41
Moderadamente entrenado/a	28452	36
Muy entrenado/a	10262	12.98
Alto nivel	393	0.5
Élite	86	0.11

Por lo tanto, se observa que la lógica aplicada tiene sentido según el porcentaje de corredores que hay en cada umbral.

2.9 Definición de la Variable de Fatiga Extrema (*Hitting the Wall*)

Para habilitar un posterior enfoque de **Aprendizaje Supervisado de Clasificación**, definimos una nueva variable objetivo binaria denominada **hit_the_wall** (coloquialmente conocido como “petar”).

Establecemos el umbral de fatiga siguiendo los estándares de la literatura científica deportiva: un atleta ha sufrido una deceleración crítica si el tiempo empleado en completar la segunda mitad de la maratón es igual o superior al **120% (un 20% más lento)** del tiempo invertido en la primera mitad.

- **Primera Mitad (T_1):** Tiempo de paso por la media maratón (**half**).
- **Segunda Mitad (T_2):** Official Time – half.
- **Condición de “Petar”:** $T_2 \geq 1.20 \times T_1 \implies \text{hit_the_wall} = 1$.


```

# 1. Calculamos el tiempo de la primera y segunda mitad en segundos
t1 = df_boston['half']
t2 = df_boston['official_time'] - df_boston['half']

# 2. Calculamos el ratio de deceleración (T2 / T1)
deceleracion_ratio = t2 / t1

# 3. Creamos la variable binaria (1 si ratio >= 1.20, 0 en caso contrario)
df_boston['hit_the_wall'] = (deceleracion_ratio >= 1.20).astype(int)

# 4. Analizamos la distribución real de cuánta gente "peta" en nuestro dataset
distribucion_muro = df_boston['hit_the_wall'].value_counts().to_frame()
distribucion_muro.columns = ['Número de Corredores']
distribucion_muro['Porcentaje (%)'] = (distribucion_muro['Número de Corredores'] / len(df_boston)) * 100
distribucion_muro.index = ['Ritmo Constante (0)', 'Chocan contra el muro (1)']

# Imprimimos la tabla resultante
print("Distribución de la variable 'hit_the_wall' en el dataset unificado:")
print(distribucion_muro.to_markdown())

```

Distribución de la variable 'hit_the_wall' en el dataset unificado:

	Número de Corredores	Porcentaje (%)
Ritmo Constante (0)	63060	79.78
Chocan contra el muro (1)	15978	20.22

3

4